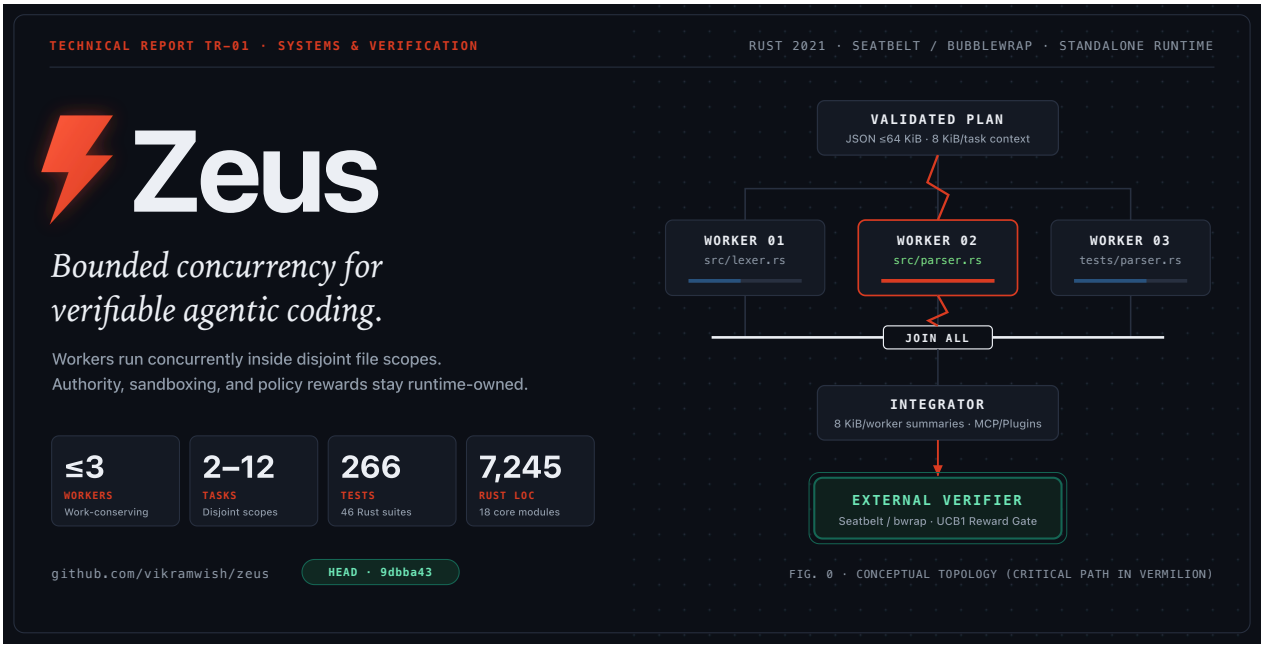
TECHNICAL REPORT TR-01 SYSTEMS, CONCURRENCY & VERIFICATION

Zeus: Bounded Concurrency and Fail-Closed Authority for Verifiable Agentic Coding

Author: Vikram Viswanath · Repository Snapshot: 9dbba43 (2026-11-13) · Implementation: 7,245 LOC Rust, 266 Tests



Abstract. Autonomous coding agents routinely conflate natural-language model output with system authority: they allow generated tool requests to escape declared paths, waste multi-core capacity behind synchronous batch barriers, or corrupt adaptive routing state whenever transient network timeouts occur. **Zeus** is a standalone Rust runtime (7,245 lines across 18 modules) that separates untrusted model reasoning from host capability enforcement, OS-level sandboxing, and verification-gated policy learning.

Zeus bounds parallel coding to 1–3 workers executing over a work-conserving FIFO queue with global pre-admission scope checking, ensuring exact-file disjointness and fail-closed rejection before any worker thread spawns. Candidate mutations exclude secret-like files and expose zero write authority over the external verifier or state directory, which run inside macOS Seatbelt or Linux `bubblewrap` profiles. Strategy selection between a direct implementer and parallel workers uses a deterministic UCB1 contextual bandit updated *strictly* on verified pass or verified fail outcomes, paired with an experimental shadow-only outcome scorer (*Zeus Judge*) governed by eight abstention gates. The implementation is verified deterministically by 266 static Rust tests across 46 suites requiring no live LLM, and evaluated across 14 exploratory local Qwen 3.5-9B field trials showing 0/10 infrastructure errors contaminating routing tables.

Reproducibility Artifacts. All figures render deterministically from Rust source & test inspection plus `research/data/field_runs.json` via `research/scripts/build_figures.py`. Companion materials: `research/index.html` (interactive lab), `research/REPORT.md`, `DESIGN.md`, and `cargo test --locked` (rev. `9dbba43`).

1. Introduction and Threat Model

Multi-step coding agents sit on an awkward fault line between stochastic token generators and deterministic operating-system side effects. When an agent edits a software repository, at least four failure modes routinely compound:

1. **Implicit capability elevation**: prompts suggest narrow tasks, while raw shell or path tools permit writes across adjacent tests, hidden secrets, or verifier harnesses.
2. **Straggler-amplified concurrency**: parallel fan-out systems stall fast workers at coarse batch barriers or allow concurrent edits to race on the same files.
3. **Biased online adaptation**: adaptive routers credit or penalize execution strategies when rate limits, deadlocked HTTP streams, or user cancellations abort a run before verification.
4. **Unbounded transcript and handoff growth**: multi-worker aggregation or multi-turn file reads blow provider context windows, causing silent truncation or mid-run abandonment.

Zeus (`github.com/vikramwish/zeus`) treats these failure modes as systems engineering invariants rather than prompt-engineering problems. Designed after direct inspection of local agent runtime ancestors (*Hermes* revisions `bf0203a7` / `c419cf7c` and *Prime* revisions `c4c238ca` / `c6d4da94`), Zeus is implemented from scratch in Rust 2021 (MSRV 1.85) with explicit dependency minimalism (`serde`, `serde_json`, `ureq`, `sha2`, `libc`, `base64`, `ring`, `url`).

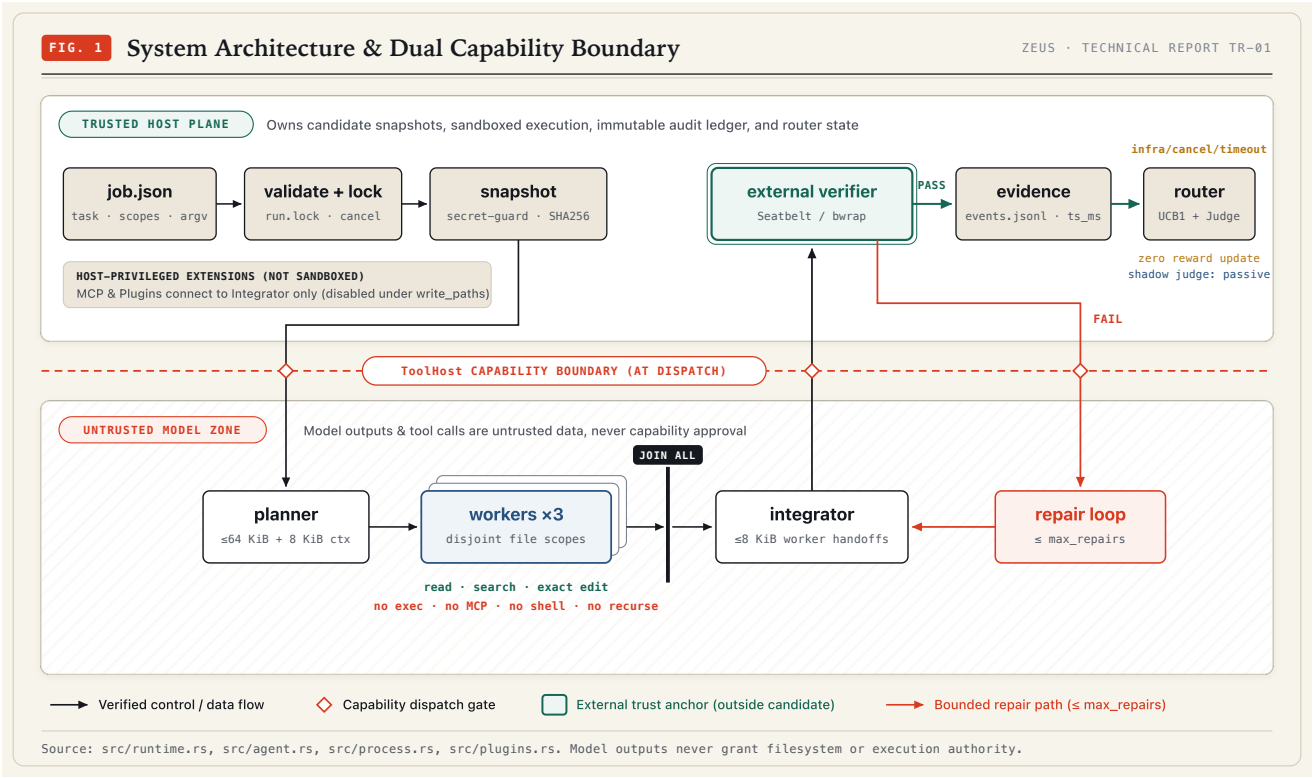


Figure 1: Zeus dual capability boundary. Host orchestration, secret-excluding candidate snapshots, OS-sandboxed external verification, immutable audit logging, and UCB1/Judge state remain inside the trusted host plane, isolated from untrusted planner, worker, and integrator completions via `ToolHost` checks.

Figure 1 illustrates the two-plane architecture. Every execution lifecycle splits cleanly at the `ToolHost` dispatch boundary:

- **Trusted Host Plane**: validates the CLI and `job.json`, acquires an exclusive `run.lock`, installs POSIX signal handlers without `SA_RESTART`, clones the repository into a candidate workspace excluding VCS/build trees and secret-like files, executes the configured external verifier inside macOS Seatbelt or Linux `bubblewrap`, persists append-only `events.jsonl`, and commits verified routing updates via atomic write-fsync-rename.

- **Untrusted Model Zone**: hosts the planner (strict JSON $\leq 64\,\text{KiB}$ plus $\leq 8\,\text{KiB}$ per-task handoffs), 1–3 concurrent file-scoped coding workers, an integrator consuming bounded 8 KiB worker handoffs, and a bounded repair loop ($\leq$ `max_repairs`, default 2, ceiling 8). No model token or tool argument is ever treated as a capability grant.

Scope of Claims and Empirical Honesty

Zeus is an early-working research prototype, not a benchmark-topping general software engineer. Deterministically passing tests verify runtime mechanics (protocol parsing, sandboxing, compaction, UCB1 math, abstention gates) against mocked servers and local fixtures. The 14 empirical field trials reflect exploratory Qwen 3.5-9B development runs on an isolated Python key-value fixture rather than a held-out benchmark. Deterministic test execution exercises both macOS Seatbelt and Linux `bubblewrap` sandboxing end-to-end in CI, whereas exploratory Qwen 3.5-9B field trials ran on macOS.

2. Formal Runtime Architecture and Invariants

2.1 Work-Conserving FIFO Admission

Rather than dispatching tasks in fixed barrier waves of size k, `agent::run_tasks` (`src/agent.rs`) maintains a shared `Mutex`-protected cursor over the validated queue $T_0, \ldots, T_{m-1}$ serviced by $k \in \{1, 2, 3\}$ scoped worker threads (`std::thread::scope`). Each idle worker atomically pops the next unclaimed index i, executes until completion or failure, and writes its bounded result directly into slot i of the output buffer.

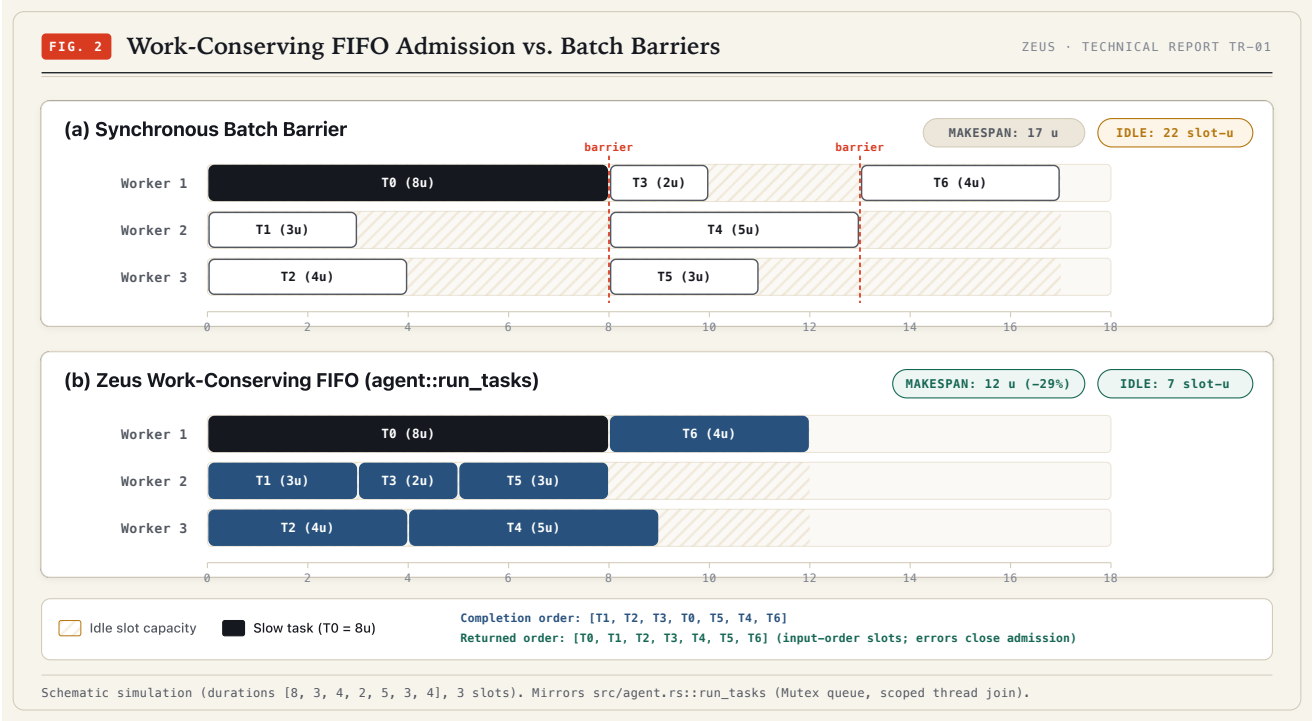


Figure 2: Comparison between synchronous batch barriers and Zeus work-conserving FIFO admission for illustrative task durations $[8, 3, 4, 2, 5, 3, 4]$ across $k = 3$ workers. Fast workers refill immediately while output slots preserve planner declaration order.

For deterministic heterogeneous workloads such as durations $d = [8, 3, 4, 2, 5, 3, 4]$ with $k = 3$ (Figure 2), a synchronous batch barrier yields makespan $\sum_b \max_{t \in b} d_t = 8 + 5 + 4 = 17\,\text{u}$ and 22 idle slot-units, whereas list-scheduled FIFO admission lowers makespan to 12 u (-29.4%) and idle capacity to 7 slot-units (-68.2%), bounded by Graham's classic greedy scheduling inequality $C_{\max} \leq \frac{1}{k}\sum d_i + \frac{k-1}{k}\max d_i$. Upon any worker error, shared cancellation state closes new queue admissions immediately while running workers finish cleanly or abort on whole-job deadline/signal checks (`src/cancel.rs`).

2.2 Pre-Admission Write-Scope Isolation

Under parallel execution (`src/runtime.rs`), Zeus invokes a planner constrained to emit 2–12 coding subtasks in a single JSON payload bounded at 64 KiB. Each subtask T_i must supply a non-empty task description, a non-empty list of explicit file claims W_i, and an optional bounded handoff `context` string. Before any worker thread spawns, Zeus evaluates five whole-plan safety invariants across $\mathcal{P} = \{(T_i, W_i)\}_{i=1}^{m}$ (denoting path prefix by $\preceq$):

Theorem 1 (Pre-Admission Disjointness & Fail-Closed Containment)

Let $p \in W_i$ be any claimed write path, let $\mathrm{norm}(p)$ denote its canonical ASCII lowercase form, and let $A \subseteq \Sigma^+$ be the optional `autonomy.write_paths` allowlist (≤ 128 entries). Plan $\mathcal{P}$ is admitted iff all five conditions hold simultaneously:

$$\begin{aligned}
&\text{(Canonical Syntax)} && \forall i, \forall p \in W_i : \ p \text{ is non-empty ASCII}, \ p_0 \neq \text{“/”}, \ \text{“..”} \notin \mathrm{segs}(p), \ \text{“//”} \notin p, && (1)\\
&\text{(Explicit Files)} && \forall i : \ W_i \neq \emptyset \ \wedge \ \forall p \in W_i, \ p \text{ names an exact file (not a directory)}, && (2)\\
&\text{(Global Disjointness)} && \forall (i,p) \neq (j,q) : \ \mathrm{norm}(p) \neq \mathrm{norm}(q) \ \wedge \ \mathrm{norm}(p) \not\preceq \mathrm{norm}(q), && (3)\\
&\text{(Autonomy Subset)} && A \neq \emptyset \implies \forall i, \forall p \in W_i : \ \exists a \in A \text{ s.t. } \mathrm{norm}(a) \preceq \mathrm{norm}(p), && (4)\\
&\text{(Bounded Handoffs)} && \forall i : \ |\mathrm{context}_i|_{\mathrm{bytes}} \leq 8192, \quad |\mathcal{P}_{\mathrm{JSON}}|_{\mathrm{bytes}} \leq 65536. && (5)
\end{aligned}$$

Violation of any clause aborts the parallel plan with zero candidate mutations and zero spawned workers.

At runtime, candidate file writes additionally acquire `Workspace.mutation: Mutex<()>` (`src/workspace.rs`), perform SHA-256 content-hash checks against pre-read state for exact-edit fidelity, and enforce per-worker tool isolation: workers expose only read, search, and exact-edit primitives—no command execution, no MCP servers, no plugin tools, and no recursive delegation. Symlinks escaping the workspace root, excluded metadata directories, and files matching `secret_like` heuristics (`PRIVATE KEY–-`, `sk-`, `AKIA`/`ASIA`, `ghp_`–`ghs_`, `xox*`, `AIza`, `.env`) are withheld before any model observes them (Figure 3).

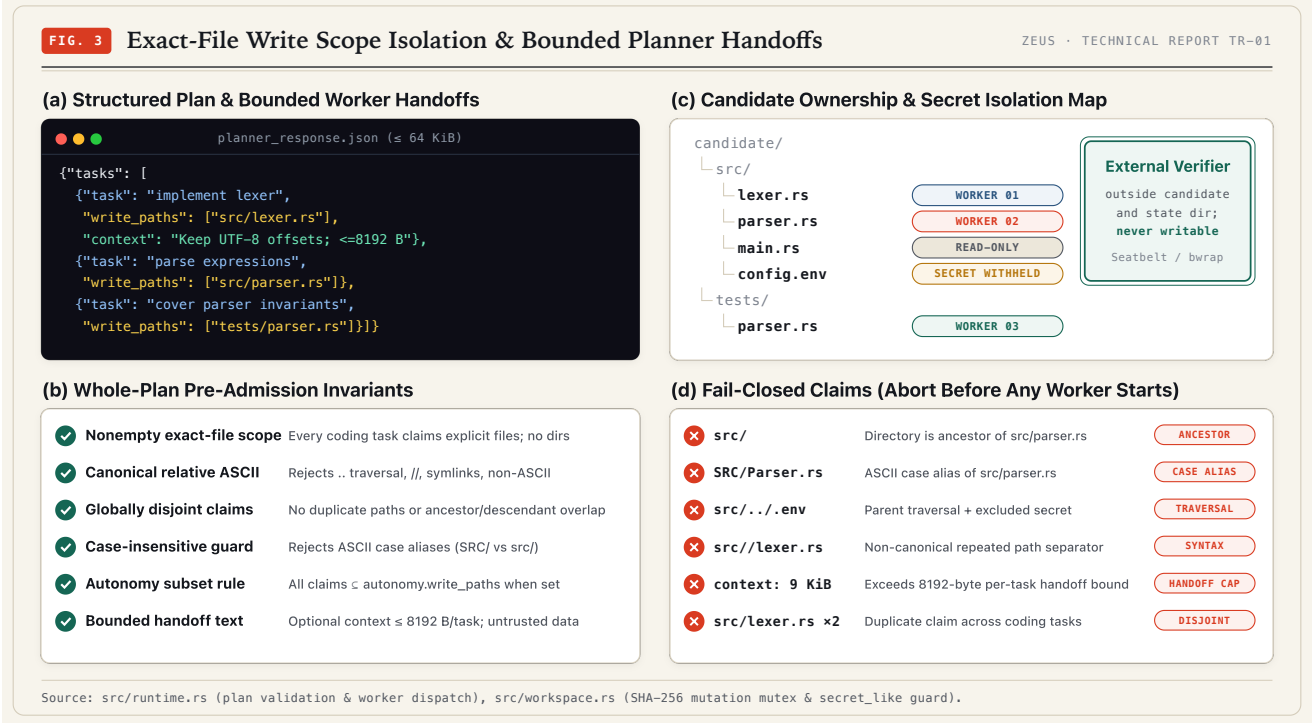


Figure 3: Whole-plan pre-admission scope verification (`src/runtime.rs`, `src/workspace.rs`). Disjoint exact-file claims proceed concurrently while ancestor, ASCII case-alias, traversal, duplicate, and oversized handoff claims abort fail-closed prior to worker spawn.

2.3 Sandboxed External Verification and Extensions Trust Model

Verification runs outside both the model loop and candidate ownership assumptions (`src/process.rs`):

- **macOS Seatbelt**: denies default capabilities and restricts writes strictly to the candidate directory and an explicit scratch path while keeping source repository and state directories read-protected/unwritable.
- **Linux** `bubblewrap`: unshares PID and IPC namespaces (`-unshare-pid -unshare-ipc`), mounts procfs, binds the candidate tree read-write and host runtime paths read-only, drops environment secrets, and sets `-die-with-parent`.
- **Host-Privileged Extensions**: optional local MCP servers (`src/mcp.rs`) and manifest-v1 plugins (`src/plugins.rs`, opt-in via `-allow-plugins`, capped at 16 plugins, 64 tools, and 64 skills) run on the host side rather than inside Seatbelt/bubblewrap. Accordingly, Zeus limits plugin and MCP exposure solely to the integrator (never sub-workers), automatically disables both when scoped `autonomy.write_paths` are active, and re-runs the external verifier after any `run_end` hook executes.

Table 1: Core runtime memory, concurrency, and extension bounds enforced in code (`HEAD 9dbba43`).

Mechanism	Module	Hard Bound	Enforcement Behavior
Worker Concurrency	`src/runtime.rs`	$1 \le k \le 3$	Work-conserving FIFO queue over 2–12 coding subtasks
Planner Response	`src/runtime.rs`	≤ 64 KiB JSON	Pre-admission validation rejects malformed or oversized plans
Per-Task Planner Context	`src/runtime.rs`	$\le 8{,}192$ B	Bounded hint string passed cleanly into worker prompt
Worker Handoff Projection	`src/output.rs`	$\le 8{,}192$ B / worker	75% head / 25% tail + `[zeus: middle omitted]`; raw output in `events.jsonl`
Transcript Compaction	`src/agent.rs`	≥ 64 KiB (1 MiB def.)	Elides old tool outputs (`{"elided":true}`); protects last 4 results
Tool Output	`src/agent.rs`	256 KiB / 128 KiB	Truncates oversized tool reads; caps `process::capture` at 128 KiB
Autonomy Guardrails	`docs/AUTONOMY.md`	≤ 128 paths / 8 repairs	Restricts write scope, repair rounds (0–8, def. 2), and calls (1–1024)
Snapshot Ingestion	`src/workspace.rs`	2,000 files / 64 MiB	Depth ≤ 128, per-file ≤ 16 MiB; skips VCS/deps and secrets
Extension Envelope	`src/plugins.rs`	16 plugins / 64 tools	Disabled under `write_paths`; triggers mandatory `run_end` re-verification

3. Verifier-Anchored Bandit Routing

For any coding context string c (≤ 256 B, up to 1,024 retained contexts per policy file saved atomically with `0o600` permissions), Zeus selects an execution strategy $a \in \{\text{Direct}, \text{Parallel}\}$ using deterministic UCB1 (`src/policy.rs`):

$$\pi(c) = \begin{cases} \text{Direct}, & n_c(\text{Direct}) = 0 \\ \text{Parallel}, & n_c(\text{Parallel}) = 0 \\ \text{Direct}, & \text{score}(\text{Direct}) = \text{score}(\text{Parallel}) \\ \arg\max_a \left[\frac{s_c(a)}{n_c(a)} + \sqrt{\frac{2\ln(n_c(\text{Direct})+n_c(\text{Parallel}))}{n_c(a)}} \right], & \text{otherwise.} \end{cases} \tag{6}$$

Crucially, UCB1 updates are gated strictly by external verification results (Figure 4). A verified pass adds $(+1 \text{ trial}, +1 \text{ success})$; a verified functional failure adds $(+1 \text{ trial}, +0 \text{ success})$; transport timeouts, planner JSON syntax errors, unknown tool calls, path rejections, and user interrupts leave $n_c(a)$ and $s_c(a)$ untouched. Without this isolation, fragile infrastructure failures on one arm would permanently inflate the empirical win rate of whichever strategy ran under healthier network conditions.

4. Empirical Field Trials and Hardening Trajectory

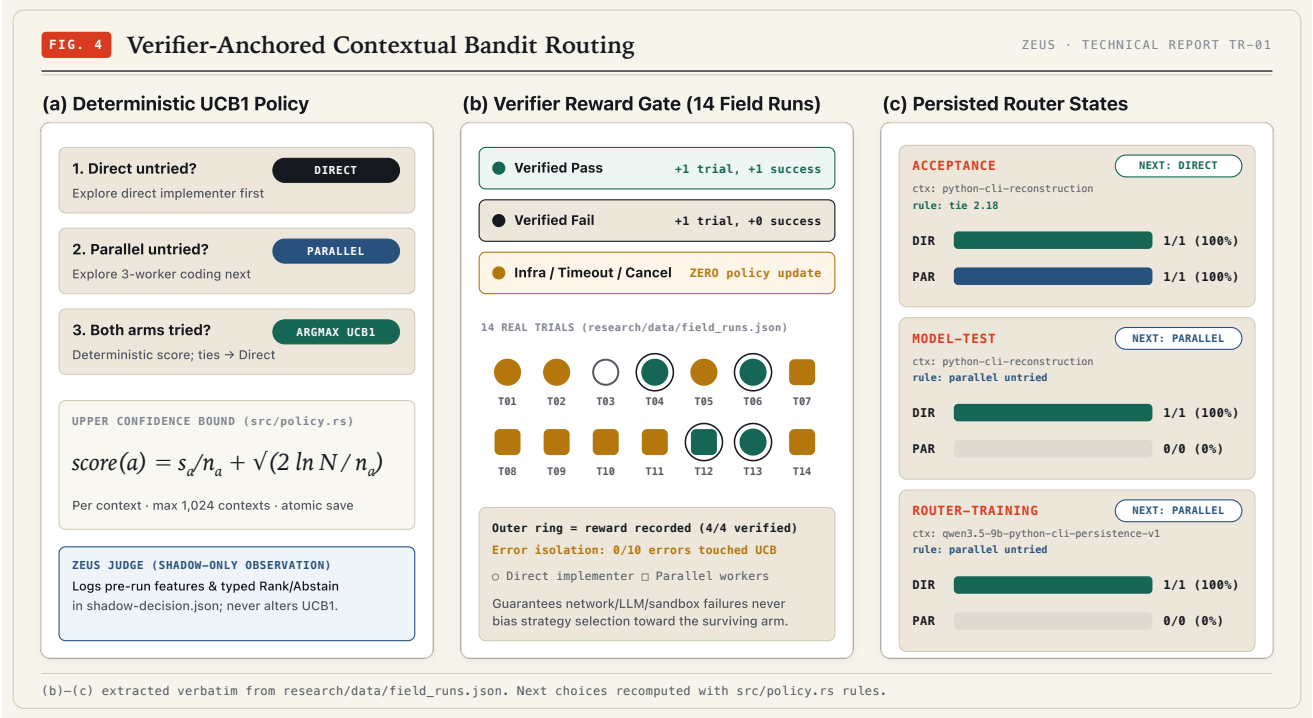


Figure 4: Deterministic UCB1 policy and verifier reward gate (`src/policy.rs`). Across all 14 empirical field trials, only verified outcomes (outer rings on $T04$, $T06$, $T12$, $T13$) updated router counts; 0/10 transport, planner-format, capability, or path-contract failures touched UCB weights.

During initial runtime construction (November 1–3), 14 exploratory local runs (`T01`–`T14`) were executed using Qwen 3.5-9B against a deterministic Python key-value store reconstruction fixture (`kv.py`, growing from 77 to 82 verification assertions). All run manifests and policy files were scrubbed of local paths into `research/data/field_runs.json`.

Figure 5 reveals two findings:

1. **Observed runtime bottlenecks drove concrete engineering commits**: early path-contract failures (`T01`) and strict JSON planner errors (`T07`, `T09`, `T10`) led to planner JSON envelopes (`d121e6b`); sluggish multi-minute model stalls led to per-request deadlines (`3267d6f`); denied-action confusion led to typed error feedback (`3f7e00a`), immediately preceding parallel success `T12` (82/82 assertions passed). Subsequent hardening (`b9d4a4a`–`e4143e4`, plus CI `bubblewrap`/FD hygiene in `9dbba43`) targeted HTTP retries, compaction, whole-job deadlines, secret withholding, shadow scoring, and sandbox boundary enforcement.
2. **Router invariance held under stress**: despite 10 incomplete/infrastructure runs out of 14 (due to Qwen 3.5-9B latency, local Ollama/LM transport timeouts, format errors, and manual cancellation on `T03`), *zero* non-verified runs wrote policy state. In `acceptance`, both arms hold 1/1 (100% verified); under the exact tie-breaking rule in `src/policy.rs` (2.18 = 2.18), Direct is deterministically chosen next, whereas `model-test` and `router-training` hold 1/1 Direct and 0/0 Parallel, cleanly selecting Parallel on the next execution.

5. Deterministic Verification Corpus

To decouple systems correctness from model availability, Zeus ships a static suite of 266 `#[test]` functions spanning 44 integration test files under `tests/` plus 3 inline unit tests in `src/runtime.rs` and 1 unit test in `src/process.rs` (46 suites total; 2 tests marked `#[ignore]` for explicit opt-in runs alongside OS-gated Seatbelt/`bubblewrap` checks).

Figure 6 details test allocation across nine subsystems, led by model protocol, retry, and compaction coverage (58 tests), workspace/secret/output guards (40), plugins/MCP/analyzers (39), sandboxes/verifier/interruption (37), and policy/judge/evidence checks (30). Dedicated process-safety tests assert clean file-descriptor hygiene, verifying that external verifiers do not inherit open host sockets or launch channels when spawned cleanly.

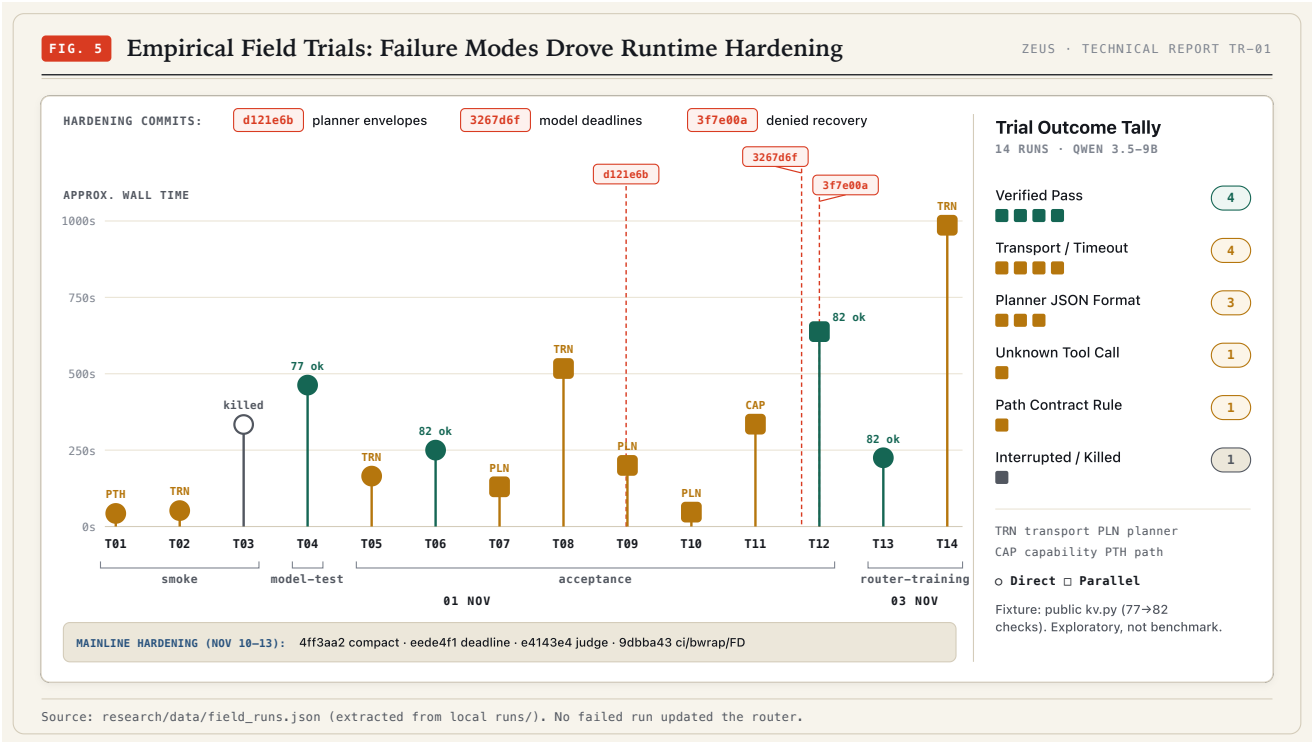


Figure 5: Chronology of all 14 exploratory Qwen 3.5-9B field trials (`T01`–`T14`) extracted verbatim from local `runs/`. Every infrastructure failure class catalyzed concrete runtime fixes rather than polluting router state.

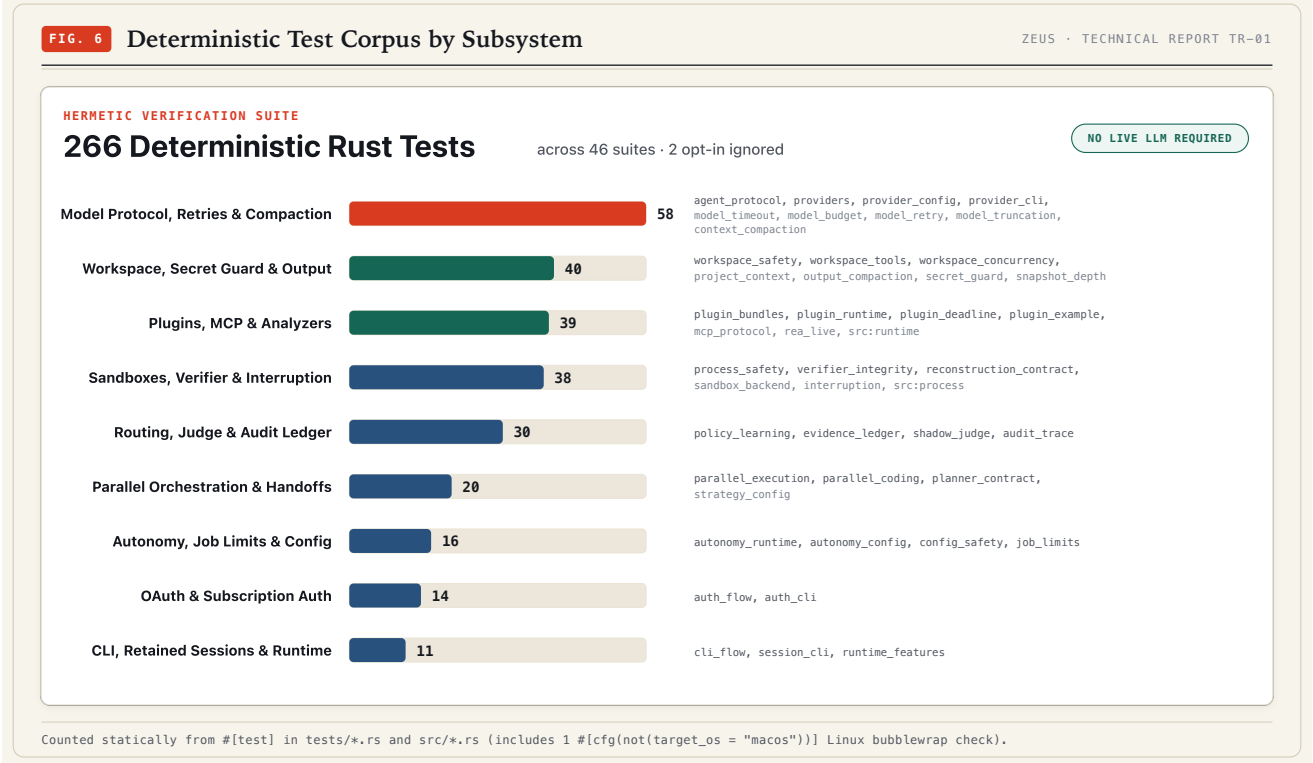


Figure 6: Static breakdown of Zeus's 266 deterministic Rust tests grouped into nine architectural subsystems on revision `9dbba43`.

6. Bounded Memory and Zeus Judge Shadow Abstention

Long coding trajectories fail abruptly when raw test logs or recursive file inspections overflow context limits (historically observed as provider rejections past ~ 4 MiB). Mainline hardening between commits `abf3e7a` and `3b84f90` introduced three deterministic safeguards summarized in Table 1:

1. **Deterministic Tool-Result Compaction** (`src/agent.rs`): once prompt bytes cross `compaction_threshold_bytes` (default 1 MiB, floor 64 KiB) or on recoverable provider overflow, Zeus replaces older tool results in-place with `{"elided":true}` while unconditionally preserving the most recent `PROTECT_LAST_TOOL_RESULTS` $= 4$ tool outputs (Figure 7, right). Because edits operate reversibly over exact JSON markers and log an audit event, the model can re-read any compacted file rather than relying on lossy LLM summarization.
2. **Bounded Handoff Projection** (`src/output.rs`): after all parallel workers join, raw worker outputs still stream verbatim to `events.jsonl`, while the integrator prompt receives `worker_handoff` capped at 8,192 B per worker using an asymmetric 75% head / 25% tail UTF-8-safe split separated by `\n[zeus: middle omitted]\n`. Even across the maximum 12 subtasks, integrator aggregation stays capped around 96 KiB.
3. **Transient Retry Classification & Whole-Job Deadlines** (`src/model.rs`, `src/cancel.rs`): HTTP transport timeouts and transient status codes trigger bounded retries with explicit classification, while whole-job deadlines and token ceilings halt runaway loops predictably across planner, worker, and integrator phases.

While UCB1 explores across coarse contexts, jobs vary substantially in task size and autonomy limits. Commit `e4143e4` introduces *Zeus Judge* (`src/judge.rs`), an experimental offline-fitted dual-head linear/logistic outcome estimator that records counterfactual recommendations to `shadow-decision.json` and post-run outcomes to `shadow-outcome.json` without altering runtime routing. Each job is projected onto an 8-dimensional vector under schema `job_limits_v1`:

$$x = \left[\frac{|\text{task}|}{32768}, \frac{\text{max_turns}}{32}, \frac{\text{max_repairs}}{8}, \frac{\text{max_calls}}{1024}, \mathbf{1}_{\text{calls}}, \frac{\ln(1+\text{max_tokens})}{\ln(1+2^{64}-1)}, \mathbf{1}_{\text{tokens}}, \frac{\text{max_job_s}}{86400}\right],$$

bound cryptographically via SHA-256($\{\text{context}, \text{model}, \text{protocol}, \text{endpoint}, \text{verifier}\}$).

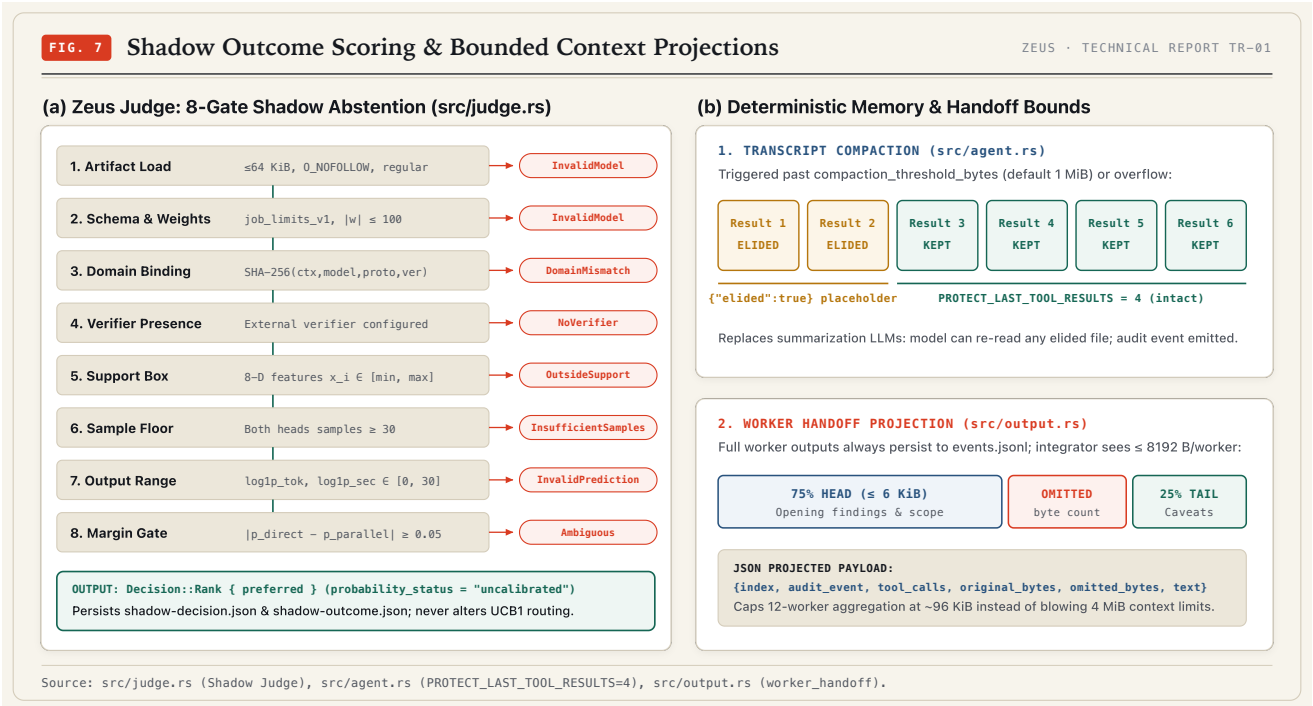


Figure 7: Left: 8-gate fail-closed abstention pipeline in `src/judge.rs`, preventing uncalibrated or out-of-domain artifacts from producing a preferred strategy rank. Right: deterministic transcript compaction and bounded worker handoff projections.

As shown in Figure 7 (left), `judge::evaluate` must clear eight sequential integrity gates before returning `Decision::Rank`: (1) artifact read via `O_NOFOLLOW` regular file ≤ 64 KiB, (2) schema `job_limits_v1` with finite weights $|w| \leq 100$, (3) exact

domain SHA-256 equality, (4) external verifier presence, (5) support box containment $x_i \in [\min_i, \max_i]$, (6) sample floor $n_{\text{samples}} \geq 30$ on both heads, (7) finite predicted $\log(1 + \text{tok})$ and $\log(1 + \text{sec})$ in $[0, 30]$, and (8) pairwise decision margin $|p_{\text{direct}} - p_{\text{parallel}}| \geq 0.05$. Even when ranked, probability outputs explicitly carry `probability_status = "uncalibrated"`.

7. Conclusion

Zeus demonstrates that reliability gaps in agentic coding stem largely from missing OS, concurrency, and contract boundaries rather than insufficiently persuasive system prompts. By keeping candidate snapshots secret-free, forcing pre-admission disjointness across at most three work-conserving workers, projecting integrator handoffs into bounded 8 KiB envelopes, isolating adaptive routing updates behind sandboxed verification, and wrapping learned outcome models inside eight fail-closed abstention gates, Zeus turns nondeterministic LLM completions into auditable, restart-safe systems behavior. Future empirical work requires held-out software benchmarks spanning stronger frontier and local models, cross-platform field trials on Linux `bubblewrap` hosts, and collecting ≥ 30 samples per head to graduate Zeus Judge from abstention into calibrated active evaluation.

References

[1] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

[2] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

[3] V. Viswanath. Zeus Implementation Contract and Ancestor Provenance (`ARCHITECTURE.md`, inspecting *Hermes* `bf0203a7`/`c419cf7c` and *Prime* `c4c238ca`/`c6d4da94`), 2026.

[4] V. Viswanath. Zeus Judge Shadow Outcome Contract (`docs/JUDGE.md`, `src/judge.rs`, rev. `9dbba43`), 2026.

[5] Project Atomic / Containers. *bubblewrap*: unprivileged sandboxing tool for Linux namespaces. https://github.com/containers/bubblewrap.